```
print("Welcome to Treasure Bot")
print("Your mission is to find the treasure.")
print("Let's find the treasure together!\n")

map_rooms = {
    "A": ["B", "C"],
    "B": ["D"],
    "C": ["D"],
    "D": ["G"],
    "G": []
}

print("Treasure Map:\n")
print("          A(Start)")
print("         / \\")
print("        B   C")
print("         \\ /")
print("          D")
print("          |")
print("          G(Treasure💎)")
print("\nDoors from each room:")
print(map_rooms)
```

```
Welcome to Treasure Bot
Your mission is to find the treasure.
Let's find the treasure together!

Treasure Map:

          A
         / \
        B   C
         \ /
          D
          |
          G(Treasure💎)

Doors from each room:
{'A': ['B', 'C'], 'B': ['D'], 'C': ['D'], 'D': ['G'], 'G': []}
```

```
def bfs(start,goal):
    to_do = [start]
    visited = []
    order = []

    while to_do:
        room = to_do.pop(0)
        if room in visited:
            continue
        visited.append(room)
        order.append(room)
        if room == goal:
            return order
        for nxt in map_rooms[room]:
            if nxt not in visited:
                to_do.append(nxt)
    return order

# Driver code
order = bfs("A", "G")

print("BFS checked the rooms in this order:", order)
print("Number of rooms BFS checked:", len(order))
```

```
BFS checked the rooms in this order: ['A', 'B', 'C', 'D', 'G']
Number of rooms BFS checked: 5
```

```
map_rooms = {
    "A": ["B", "C"],
    "B": ["D"],
    "C": ["D"],
    "D": ["G"],
    "G": []
}
def dfs(start,goal):
```

```
    to_do = [start]
    visited = []
    order = []

    while to_do:
        room = to_do.pop()
        if room in visited:
            continue
        visited.append(room)
        order.append(room)
        if room == goal:
            return order
        for nxt in map_rooms[room]:
            if nxt not in visited:
                to_do.append(nxt)
    return order

# Driver code
order = dfs("A", "G")

print("DFS checked the rooms in this order:", order)
print("Number of rooms DFS checked:", len(order))
```

```
DFS checked the rooms in this order: ['A', 'C', 'D', 'G']
Number of rooms DFS checked: 4
```

```
# Map of rooms
map_rooms = {
    "A": ["B", "C"],
    "B": ["D"],
    "C": ["D"],
    "D": ["G"],
    "G": []
}


# Breadth-First Search (BFS)
def bfs(start, goal):
    to_do = [start]
    visited = []
    order = []

    while to_do:
        room = to_do.pop(0)

        if room in visited:
            continue

        visited.append(room)
        order.append(room)

        if room == goal:
            return order

        for nxt in map_rooms[room]:
            if nxt not in visited and nxt not in to_do:
                to_do.append(nxt)

    return order


# Depth-First Search (DFS)
def dfs(start, goal):
    to_do = [start]
    visited = []
    order = []

    while to_do:
        room = to_do.pop()   # Pop from end for DFS

        if room in visited:
            continue

        visited.append(room)
        order.append(room)

        if room == goal:
```

```
            return order

        for nxt in map_rooms[room]:
            if nxt not in visited and nxt not in to_do:
                to_do.append(nxt)

    return order


# Door costs
door_costs = {
    ("A", "B"): 1,
    ("B", "D"): 1,
    ("A", "C"): 5,
    ("C", "D"): 1,
    ("D", "G"): 1
}


# Function to calculate path cost
def path_cost(path):
    total = 0

    for i in range(len(path) - 1):
        door = (path[i], path[i + 1])

        if door not in door_costs:
            print(f"Warning: Door {door} not found.")
            return float("inf")

        total += door_costs[door]

    return total


# Define two possible paths
path1 = ["A", "B", "D", "G"]
path2 = ["A", "C", "D", "G"]

print("Calculating path costs:")
print("Path 1 A -> B -> D -> G costs:", path_cost(path1))
print("Path 2 A -> C -> D -> G costs:", path_cost(path2))

cost1 = path_cost(path1)
cost2 = path_cost(path2)

# Determine cheapest path
if cost1 < cost2:
    best_path = path1
    best_cost = cost1
else:
    best_path = path2
    best_cost = cost2

print("Path 1 cost:", cost1)
print("Path 2 cost:", cost2)

print("The cheapest path is:", best_path, "with cost", best_cost)

# BFS and DFS
bfs_order = bfs("A", "G")
dfs_order = dfs("A", "G")

print("\nRESULTS")
print("-" * 40)

print("BFS (nearest first) checked:", len(bfs_order), "rooms:", bfs_order)
print("DFS (deep first) checked:", len(dfs_order), "rooms:", dfs_order)
print("Cheapest path:", best_path, "with cost", best_cost)

print("-" * 40)

print("What we learned:")
print("BFS checks the nearest rooms first (spreads out).")
print("DFS dives deep down one path first.")
print("Cheapest path counts door costs and picks the lowest total.")
```

```
# Change door costs
door_costs[("A", "B")] = 4
door_costs[("A", "C")] = 1

cost1 = path_cost(path1)
cost2 = path_cost(path2)

print("After changing door costs:")
print("Now Path 1 (through B) costs:", cost1)
print("Now Path 2 (through C) costs:", cost2)

if cost1 < cost2:
    print("Cheapest path is now: A ---> B ---> D ---> G")
else:
    print("Cheapest path is now: A ---> C ---> D ---> G")
```

```
Calculating path costs:
Path 1 A -> B -> D -> G costs: 3
Path 2 A -> C -> D -> G costs: 7
Path 1 cost: 3
Path 2 cost: 7
The cheapest path is: ['A', 'B', 'D', 'G'] with cost 3

RESULTS
----------------------------------------
BFS (nearest first) checked: 5 rooms: ['A', 'B', 'C', 'D', 'G']
DFS (deep first) checked: 4 rooms: ['A', 'C', 'D', 'G']
Cheapest path: ['A', 'B', 'D', 'G'] with cost 3
----------------------------------------
What we learned:
BFS checks the nearest rooms first (spreads out).
DFS dives deep down one path first.
Cheapest path counts door costs and picks the lowest total.
After changing door costs:
Now Path 1 (through B) costs: 6
Now Path 2 (through C) costs: 3
Cheapest path is now: A ---> C ---> D ---> G
```